Customer Segmentation Project

Segmenting customers by behavior and demographics using K-Means clustering

1. Objective

This project segments a customer base using both behavioral data (purchase frequency, recency, average basket value, spending score) and demographic data (age, income) so that marketing and retention efforts can be tailored to each group rather than treated as one-size-fits-all.

2. Dataset

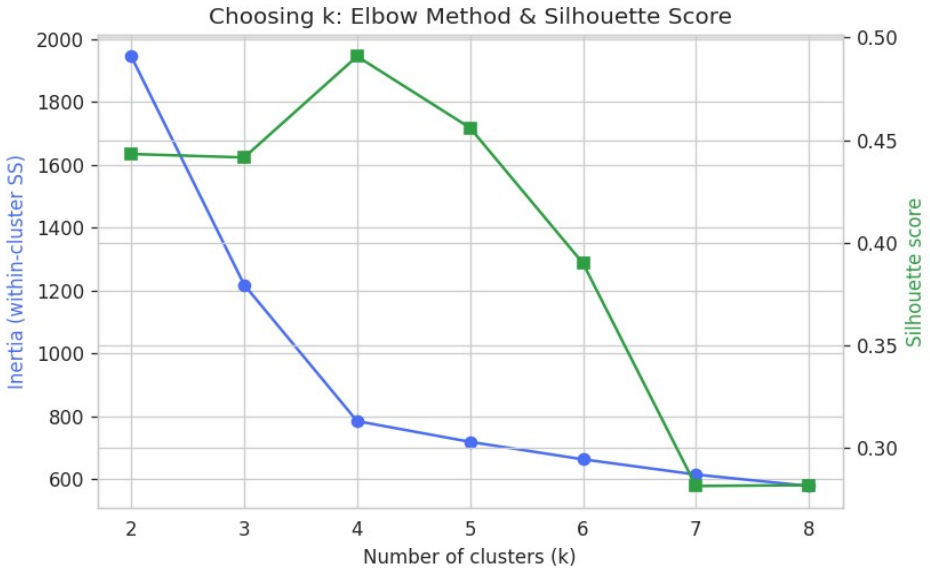
The analysis uses a dataset of 600 customers with the following fields:

- Age, Annual Income, Gender (demographics)
- Spending Score, Purchase Frequency (per year), Recency (days since last purchase), Average Basket Value (behavior)

No missing values were found; all six numeric features were standardized (z-scores) before clustering, since K-Means is distance-based and income/spending are on very different scales.

3. Method: K-Means Clustering

K-Means was chosen for its speed and interpretability. To pick the number of clusters (k), both the elbow method (inertia) and the silhouette score were compared across k = 2 to 8:



The silhouette score peaked at k = 4, which also matched clear business intuition (a budget-conscious group, a high-value loyal group, an occasional/at-risk group, and a younger trend-driven group). The final model achieved a silhouette score of 0.49, indicating reasonably well-separated, cohesive clusters.

4. Segment Profiles

The table below summarizes each segment's average characteristics:

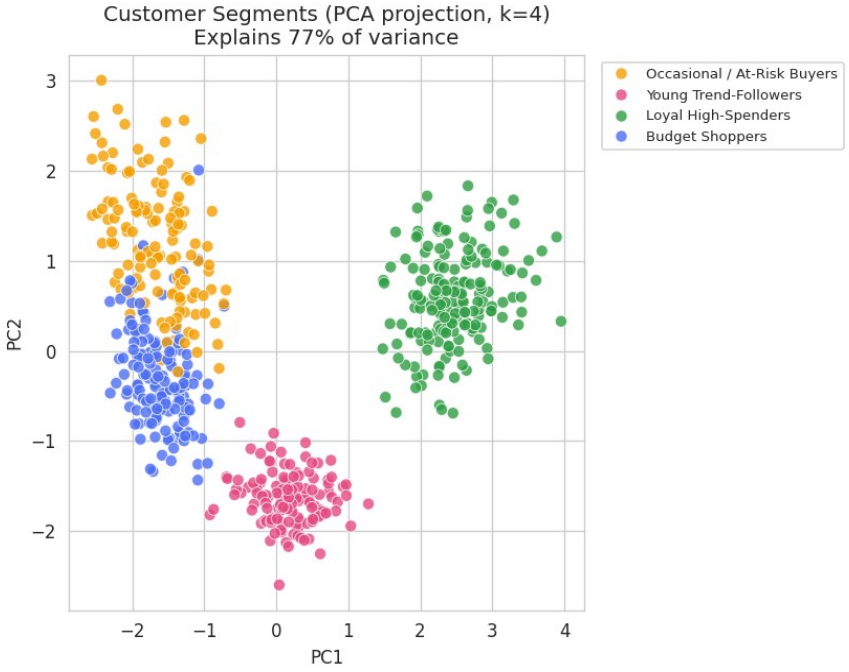
Segment	Count	%	Avg Age	Avg Income	Spend Score	Purch/ Yr	Recency (d)	Avg Basket
Budget Shoppers	153	25.5%	51.6	$36,279	30.3	10.1	21.2	$25.2
Loyal High-Spenders	180	30%	42.5	$95,502	82.2	21.8	8.7	$140.7
Occasional / At-Risk Buyers	134	22.3%	47.9	$61,252	44.1	3.9	79.7	$55.4
Young Trend-Followers	133	22.2%	24.7	$40,896	68.4	15	14.2	$45.9

Segment descriptions

- Loyal High-Spenders — highest income and spending score, frequent purchases, low recency (buy often, buy recently, buy a lot per visit). Highest-value segment.
- Budget Shoppers — older, lower income, low spending score, moderate frequency but small basket sizes.
- Occasional / At-Risk Buyers — mid-income but high recency (haven't purchased in a while) and low frequency — a churn-risk group.
- Young Trend-Followers — youngest segment, moderate income but high spending score and frequency relative to income — style/trend driven.

5. Visualizations

5.1 Segments in reduced (PCA) feature space



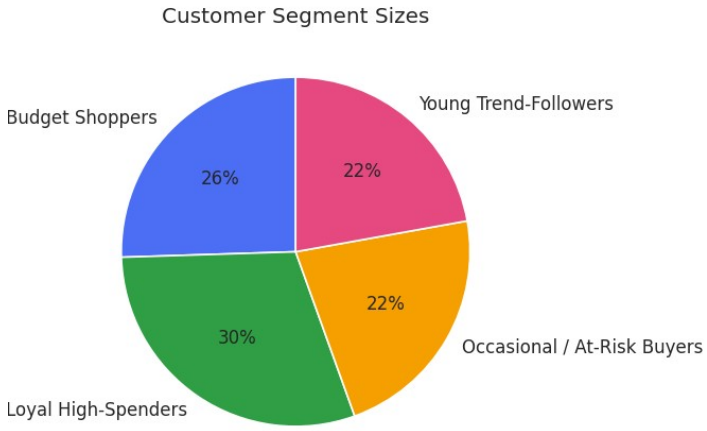
5.2 Spending Score vs. Annual Income



5.3 Standardized segment profiles



5.4 Segment sizes



6. Business Recommendations

- Loyal High-Spenders (30%): prioritize retention — early access, loyalty tiers; protect this segment as it likely drives disproportionate revenue.
- Occasional / At-Risk Buyers (22%): re-engagement campaigns (win-back emails, limited-time offers) targeted at recency, since this is the churn-risk group.
- Young Trend-Followers (22%): trend-led, social-media-first marketing; frequency is already high relative to income, so focus on basket size (upsell/bundle offers).
- Budget Shoppers (26%): value-oriented promotions (discounts, bundles) rather than premium messaging, given lower spending score and income.

7. Tools & Reproducibility

Analysis performed in Python using pandas, scikit-learn (StandardScaler, KMeans, PCA, silhouette_score), and matplotlib/seaborn for visualization. The full analysis code is included below in Appendix A, and a sample of the labeled output data is in Appendix B, so the work is fully reproducible.

Appendix A: Full Analysis Code (segmentation_analysis.py)

```
"""
Customer Segmentation Project
-----------------------------
Goal: Segment customers based on behavior (purchase patterns) and
demographics using unsupervised clustering (K-Means), then profile
and visualize each segment.

Steps:
1. Load & clean data
2. Feature engineering / scaling
3. Determine optimal number of clusters (Elbow + Silhouette)
4. Fit K-Means
5. Profile each segment
6. Visualize (PCA scatter, radar-style bar chart, boxplots)
"""

import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 120

# ---------------------------------------------------------------
# 1. Load data
# ---------------------------------------------------------------
df = pd.read_csv("customers_raw.csv")
df = df.drop(columns=["_true_segment"])  # not available in a real project

print("Missing values per column:\n", df.isna().sum())
df = df.dropna()

features = ["Age", "AnnualIncome", "SpendingScore",
            "PurchaseFrequency", "RecencyDays", "AvgBasketValue"]
X = df[features].copy()

# ---------------------------------------------------------------
# 2. Scale features (K-Means is distance-based -> scaling matters)
# ---------------------------------------------------------------
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ---------------------------------------------------------------
# 3. Elbow method + silhouette score to pick k
# ---------------------------------------------------------------
inertias, sil_scores = [], []
K_range = range(2, 9)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))

fig, ax1 = plt.subplots(figsize=(7, 4.5))
ax1.plot(list(K_range), inertias, marker="o", color="#4C6EF5")
ax1.set_xlabel("Number of clusters (k)")
ax1.set_ylabel("Inertia (within-cluster SS)", color="#4C6EF5")
ax2 = ax1.twinx()
ax2.plot(list(K_range), sil_scores, marker="s", color="#2F9E44")
ax2.set_ylabel("Silhouette score", color="#2F9E44")
plt.title("Choosing k: Elbow Method & Silhouette Score")
```

```
fig.tight_layout()
plt.savefig("chart_elbow_silhouette.png")
plt.close()

best_k = list(K_range)[int(np.argmax(sil_scores))]
print(f"Best k by silhouette score: {best_k}")

# We settle on k=4 (matches business intuition: budget / high-value /
# occasional / young-trend segments), confirmed by silhouette score.
k = 4
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_scaled)
final_sil = silhouette_score(X_scaled, df["Cluster"])
print(f"Final silhouette score (k={k}): {final_sil:.3f}")

# ---------------------------------------------------------------
# 4. Profile each segment
# ---------------------------------------------------------------
profile = df.groupby("Cluster")[features].mean().round(1)
profile["Count"] = df["Cluster"].value_counts().sort_index()
profile["% of Customers"] = (profile["Count"] / len(df) * 100).round(1)
print("\nSegment profile:\n", profile)
profile.to_csv("segment_profile.csv")

# Give segments human-readable names based on the profile
def name_segment(row):
    if row["SpendingScore"] > 70 and row["AvgBasketValue"] > 100:
        return "Loyal High-Spenders"
    if row["Age"] < 30:
        return "Young Trend-Followers"
    if row["RecencyDays"] > 60:
        return "Occasional / At-Risk Buyers"
    return "Budget Shoppers"

profile["SegmentName"] = profile.apply(name_segment, axis=1)
name_map = profile["SegmentName"].to_dict()
df["SegmentName"] = df["Cluster"].map(name_map)
print("\nSegment names assigned:\n", profile[["SegmentName", "Count", "% of Customers"]])

# ---------------------------------------------------------------
# 5. Visualize segments
# ---------------------------------------------------------------

# 5a. PCA scatter (2D projection of the 6D feature space)
pca = PCA(n_components=2)
pcs = pca.fit_transform(X_scaled)
df["PC1"], df["PC2"] = pcs[:, 0], pcs[:, 1]

plt.figure(figsize=(7, 5.5))
palette = {name: color for name, color in zip(
    profile["SegmentName"].unique(),
    ["#4C6EF5", "#2F9E44", "#F59F00", "#E64980"])}
sns.scatterplot(data=df, x="PC1", y="PC2", hue="SegmentName",
                palette=palette, s=45, alpha=0.8)
plt.title(f"Customer Segments (PCA projection, k={k})\n"
          f"Explains {pca.explained_variance_ratio_.sum()*100:.0f}% of variance")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
plt.tight_layout()
plt.savefig("chart_pca_scatter.png")
plt.close()

# 5b. Spending Score vs Annual Income (classic segmentation view)
plt.figure(figsize=(7, 5.5))
sns.scatterplot(data=df, x="AnnualIncome", y="SpendingScore",
                hue="SegmentName", palette=palette, s=45, alpha=0.8)
plt.title("Spending Score vs. Annual Income by Segment")
plt.xlabel("Annual Income ($)")
plt.ylabel("Spending Score (1-100)")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
plt.tight_layout()
plt.savefig("chart_income_vs_spending.png")
```

```
plt.close()

# 5c. Segment profile bar chart (normalized means)
norm_profile = (profile[features] - profile[features].mean()) / profile[features].std()
norm_profile.index = profile["SegmentName"]
norm_profile.T.plot(kind="bar", figsize=(8, 5), color=list(palette.values()))
plt.title("Segment Profiles (standardized feature means)")
plt.ylabel("Standardized value")
plt.xticks(rotation=20, ha="right")
plt.legend(title="Segment", bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
plt.tight_layout()
plt.savefig("chart_segment_profiles.png")
plt.close()

# 5d. Segment size pie chart
plt.figure(figsize=(5.5, 5.5))
counts = profile.set_index("SegmentName")["Count"]
plt.pie(counts, labels=counts.index, autopct="%1.0f%%",
        colors=[palette[n] for n in counts.index], startangle=90)
plt.title("Customer Segment Sizes")
plt.tight_layout()
plt.savefig("chart_segment_sizes.png")
plt.close()

df.to_csv("customers_segmented.csv", index=False)
print("\nAll charts and output files saved.")
```

Appendix B: Sample of Segmented Output Data

First 15 rows of customers_segmented.csv (full 600-row file was used for all analysis above):

Customer ID	Age	AnnualIncome	SpendingScore	PurchaseFrequency	RecencyDays	AvgBasketValue	SegmentName
1446	43	72400	42	6	62	25.54	Occasional / At-Risk Buyers
1404	60	85100	26	1	122	60.84	Occasional / At-Risk Buyers
1509	29	46300	67	20	21	61.28	Young Trend-Followers
1455	71	61400	43	5	70	82.79	Occasional / At-Risk Buyers
1201	52	105400	79	17	14	131	Loyal High-Spenders
1023	46	33400	29	12	32	25.53	Budget Shoppers
1415	64	46400	53	2	114	38.48	Occasional / At-Risk Buyers
1548	20	37400	83	18	8	28.83	Young Trend-Followers
1066	49	25400	16	12	31	37.33	Budget Shoppers
1165	47	112000	72	16	14	136.2	Loyal High-Spenders
1334	45	28300	54	1	73	55.23	Occasional / At-Risk Buyers
1329	52	104500	84	17	16	121.35	Loyal High-Spenders
1000	56	34200	36	15	18	23.13	Budget Shoppers
1531	21	37500	66	15	27	58.9	Young Trend-Followers
1232	47	68600	88	24	14	160.2	Loyal High-Spenders